AI Chat Application: Interview Guide

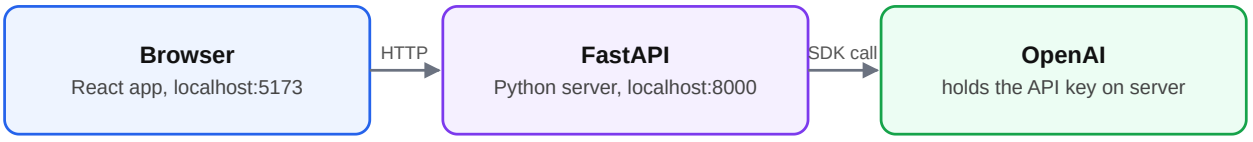
React + TypeScript + Vite frontend, FastAPI + Pydantic backend, OpenAI-compatible model.

27 interview questions with model answers and diagrams, one section per phase.

Contents

Phase 1: Project setup
Phase 2: React UI with mocks
Phase 3: Hardcoded endpoint
Phase 4: Pydantic validation
Phase 5: Fetch and CORS
Phase 6: OpenAI integration
Phase 7: History, limits and retry
Phase 8: Testing and provider flexibility
Phase 9: Project-level questions

Phase 1: Project setup



Key lives only on the middle box. The browser never receives it.

Q1.1 Why must the OpenAI call go through a backend instead of the browser?

> Anything in browser code is public. Anyone can open devtools or read the bundle and copy the key, then run up your bill. The backend holds the key in an environment variable, and the browser only calls your own API.

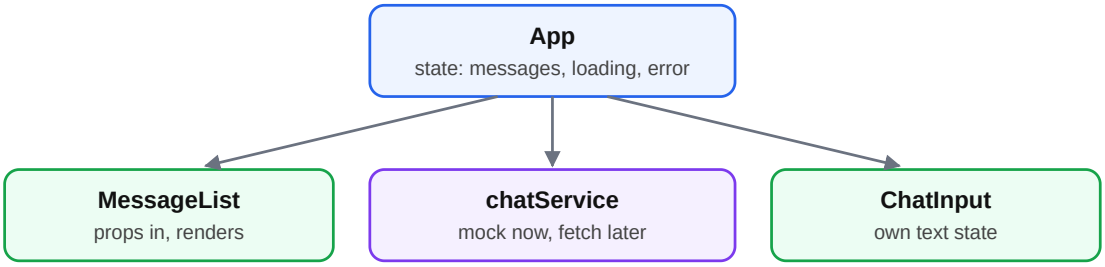
Q1.2 What is a Python virtual environment, and what is the Node equivalent?

> A virtual environment is an isolated Python install with its own packages, so projects don't collide. It's like a local node_modules, but it also holds its own interpreter. Without one, pip installs globally.

Q1.3 What does `uvicorn main:app` mean; how do FastAPI and Uvicorn relate?

> It means: in module main.py, use the object named app. FastAPI is the framework that defines routes and builds the app. Uvicorn is the ASGI server that listens on a port and runs it. Like Express (framework) and Node's http server (runtime).

Phase 2: React UI with mocks



Q2.1 Why compute the new array first instead of calling setMessages then reading messages?

> State updates are asynchronous and batched. Reading messages right after setMessages still gives the old array. Building history = [...messages, userMsg] first gives a value you can use immediately, both for the UI and for the API call.

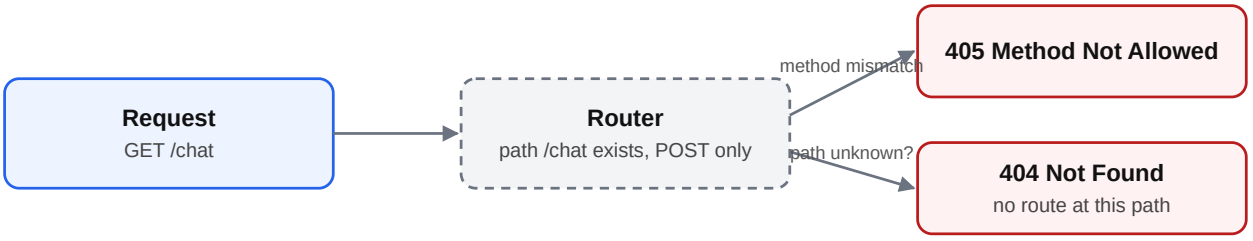
Q2.2 Why separate chatService.ts from the components?

> Components only render and emit events. The service owns the network. Swapping the mock for a real fetch touched one file, and no component changed.

Q2.3 How do you prevent a double submit while a request is in flight?

> Two guards: the button is disabled while loading (the disabled prop), and submit() re-checks canSend, so pressing Enter can't bypass the button. A state flag is the source of truth, not the DOM.

Phase 3: Hardcoded endpoint



Q3.1 Why does GET /chat return 405 instead of 404?

> 405 means the path exists but this method isn't allowed for it. 404 means no route matches the path at all. FastAPI knows /chat exists as a POST route, so it reports the method mismatch.

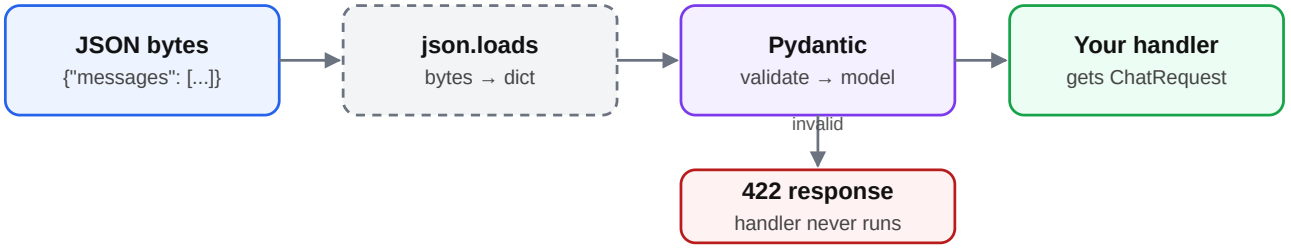
Q3.2 When would you use def vs async def in FastAPI?

> A def route runs in a thread pool, which suits blocking work like the synchronous OpenAI client. async def runs on the event loop and must never call blocking code, because that freezes every request. Use async def only with async clients.

Q3.3 How does FastAPI generate /docs without documentation code?

> It reads the route decorators and type hints (including Pydantic models), builds an OpenAPI schema, and serves Swagger UI on top of it. The code is the documentation.

Phase 4: Pydantic validation



Q4.1 How does a JSON body become a Python object in FastAPI?

> FastAPI reads the body and parses the JSON into a dict. Because the parameter is typed as a Pydantic model, it validates and converts the dict into a ChatRequest instance. Invalid data returns 422 with field-level errors, and the handler never runs.

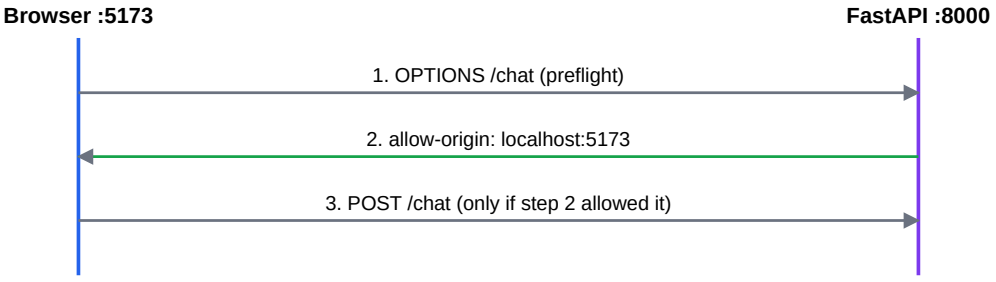
Q4.2 Why validate on the backend when the frontend already validates?

> Frontend checks are for user experience. Anyone can call the API with curl and skip the UI. The server is the only enforcement point, and it protects both cost and data quality.

Q4.3 Why Literal["user","assistant"] for role, and why reject "system"?

> Literal limits the field to exact values, like a TypeScript union of string literals. Rejecting system stops a client from injecting instructions. The server owns the system prompt, so clients can't override it.

Phase 5: Fetch and CORS



Q5.1 What is CORS, who enforces it, and what is a preflight request?

> CORS (Cross-Origin Resource Sharing) is a rule the browser enforces. A page on one origin can read a response from another origin only if that server allows it in its headers. A preflight is an OPTIONS request the browser sends first for non-simple requests, such as JSON POSTs. curl and servers aren't bound by CORS.

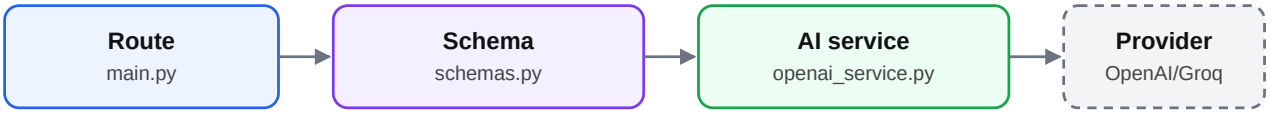
Q5.2 Why does fetch not reject on HTTP 4xx/5xx, and how do you handle that?

> fetch only rejects on network failure. An HTTP error is still a valid response. Check response.ok, read the body (FastAPI puts errors in detail), and throw an Error with a readable message.

Q5.3 Why not allow_origins=["*"], and how would you configure this for production?

> A wildcard lets any website's JavaScript call your API from a visitor's browser. Use an exact list of HTTPS origins, from an environment variable. CORS isn't authentication, so add auth before opening the API to real users.

Phase 6: OpenAI integration



Errors are raised as ChatServiceError, then mapped to HTTP status codes in the route.

Q6.1 Why keep the API key on the server, and how do env vars, .env and .gitignore work together?

> .env holds the key on your machine. load_dotenv() loads it into the process environment, and .gitignore keeps the file out of commits. The browser never sees it. In production, the platform's secret store replaces .env. Vite's VITE_ prefix is public, so secrets never use it.

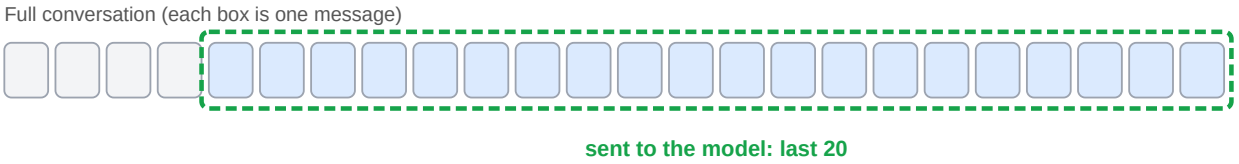
Q6.2 What is a system prompt versus user/assistant messages, and why must the server own it?

> The system prompt sets the assistant's rules and behavior. User and assistant messages are the conversation. The server owns the system prompt so clients can't change the rules. Prompt injection can still affect a model, so the system prompt isn't a security boundary by itself.

Q6.3 How do you handle OpenAI failures without leaking internals, and which status codes do you map them to?

> Catch specific SDK exceptions and map each to a status and safe message: timeout 504, connection 502, rate limit 429, bad key or missing config 500, other provider errors 502. Log the details on the server only. The order matters, because APITimeoutError is a subclass of APIConnectionError.

Phase 7: History, limits and retry



Older messages stay visible in the UI but are not sent, so the model forgets them.

Q7.1 Why trim conversation history before sending, and what are the trade-offs?

> Each turn re-sends the whole history, so cost and latency grow, and providers have context limits. A sliding window of 20 bounds both. The trade-off is that the model forgets older turns. Summaries or retrieval (RAG) are the usual next step.

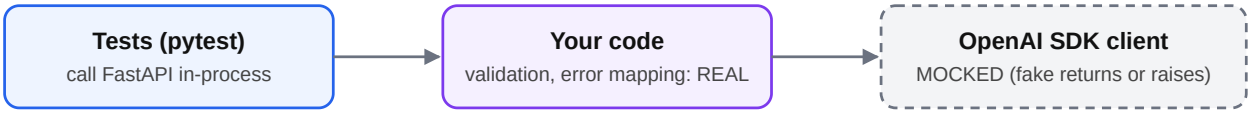
Q7.2 Why does the chat UI keep the user message after a failed request, and how does Retry avoid duplicates?

> The user's input is valid and shouldn't be lost. Retry re-sends the existing messages array, so it doesn't append the message again. State stays consistent, and the user doesn't have to retype.

Q7.3 Why must input limits exist on both frontend and backend, and which one is the real security boundary?

> The frontend limit is for honest users and gives immediate feedback. The backend limit is the boundary, because any client can bypass the UI. Keep the two constants in sync.

Phase 8: Testing and provider flexibility



Mock the network edge. Never mock the logic you are testing.

Q8.1 How do you unit-test code that calls an external API, and what should and should not be mocked?

> Mock the external boundary (the SDK client or HTTP layer) and run your own code for real. Test validation, error mapping and CORS through the real app with TestClient. Assert that invalid input never calls the mock. Don't mock the logic under test, or the test proves nothing. Keep a few real-provider checks for manual testing.

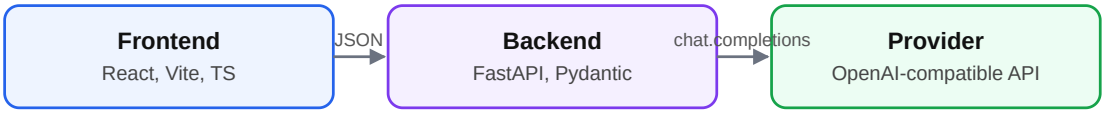
Q8.2 Why use the Chat Completions format with a configurable base URL, and what does that buy you?

> Chat Completions is the most widely implemented format, and many providers (Groq and others) implement it. Changing base_url and the model in config switches providers with no code change. The trade-off is that newer API features may not be available everywhere.

Q8.3 How would you debug a request that works in Swagger but fails in the browser?

> Swagger and curl bypass CORS, so suspect CORS first: check the preflight and the allow-origin header in devtools. Then compare the exact request: Content-Type, body, and URL, including the port. Check that the frontend uses the right API URL, and check the server log for the real error.

Phase 9: Project-level questions



Next phase (RAG) adds a retrieval step between Backend and Provider.

Q9.1 Explain the whole architecture end to end in two minutes.

> The React UI builds a message history and sends it with fetch to POST /chat. FastAPI validates it with Pydantic, the service calls the OpenAI-compatible model with a server-owned system prompt, and the reply returns as JSON. Errors are mapped to safe HTTP codes, and the key never leaves the server.

Q9.2 How would you deploy this to production?

> Put the backend on a host with secret management and no .env file. Serve the built frontend from a CDN or the same domain. Restrict CORS to the real origin, add authentication and per-user rate limits, add logging and monitoring, and consider a persistent store for conversations.

Q9.3 What would you change to add RAG (retrieval-augmented generation)?

> Add an ingestion step that splits documents into chunks and stores embeddings in a vector store. At query time, retrieve the top chunks for the user's question, add them to the prompt as context with source citations, and generate the reply. Chunking, retrieval quality and token budget would be the main design decisions.